

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет прикладной информатики

Мобильная разработка (Android и IOs)

Отчет по курсовому проекту

Выполнила:

студентка группы К3444

Кибенко Дарья Владимировна

Проверил:

Шатинский Григорий Сергеевич

Санкт-Петербург

2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 Цель и задачи курсового проекта.....	4
2 Этапы курсового проекта.....	5
2.1 Лабораторная работа №1.....	5
2.1.1 Цель лабораторной работы.....	5
2.1.2 Стэк.....	5
2.1.3 Функциональные требования.....	5
2.1.4 Нефункциональные требования.....	5
2.1.5 Выполнение лабораторной работы.....	5
2.1.5.1 Главная активность - MainActivity.....	6
2.1.5.2 Экран новостной ленты - FeedFragment.....	7
2.1.5.3 Экран профиля - ProfileFragment.....	7
2.1.5.4 Экран настроек - SettingsFragment.....	7
2.1.5.5 Вспомогательный класс логирования - Logger.....	7
2.1.5.6 Разметка и XML-ресурсы приложения.....	7
2.2 Лабораторная работа №2.....	10
2.2.1 Цель лабораторной работы.....	10
2.2.2 Функциональные требования.....	10
2.2.3 Нефункциональные требования.....	10
2.2.4 Выполнение лабораторной работы.....	10
2.2.4.1 ViewModel для хранения состояния.....	11
2.2.4.2 Экран профиля – ProfileFragment.....	11
2.2.4.3 Экран настроек – SettingsFragment.....	11
2.2.4.4 Разделение ответственности (MVVM).....	12
2.2.4.5 Логирование жизненного цикла ViewModel.....	12
2.2.4.6 Изменения в XML-ресурсах.....	12
2.3 Лабораторная работа №3.....	13
2.3.1 Цель лабораторной работы.....	13
2.3.2 Функциональные требования.....	13
2.3.3 Нефункциональные требования.....	14
2.3.4 Выполнение лабораторной работы.....	14
2.3.4.1 Изменения в слое данных.....	14
2.3.4.2 Изменения в UI-слое.....	15
2.3.4.3 Локальная база данных Room.....	17
2.3.4.4 Новый файл - MessageRepository.kt.....	18
2.3.4.5 Изменения в UI-слое.....	19
2.3.4.6 Роли компонентов.....	21
2.3.4.7 Изменения в XML-ресурсах.....	23
2.4 Лабораторная работа №4.....	24
2.4.1 Цель лабораторной работы.....	24
2.4.2 Функциональные требования.....	24
2.4.3 Нефункциональные требования.....	25

2.4.4	Выполнение лабораторной работы.....	25
2.4.4.1	Кастомный список сообщений.....	25
2.4.4.2	Применение компонентов Material Design.....	26
2.4.4.3	Поддержка лайков сообщений.....	26
2.4.4.4	Фоновая синхронизация через WorkManager.....	26
2.4.4.5	Уведомления о новых данных.....	27
2.4.4.6	Запрос разрешений и работа в онлайн/офлайн-режиме.....	28
2.4.4.7	Навигация и главный экран приложения.....	28
2.4.4.8	Изменения в UI-слое.....	29
3	Демонстрация интерфейса приложения.....	32
3.1	Экран ленты сообщений.....	32
3.2	Экран настроек.....	33
3.3	Экран профиля пользователя.....	35
	ЗАКЛЮЧЕНИЕ.....	37

ВВЕДЕНИЕ

Развитие мобильных технологий и повсеместное распространение смартфонов привели к росту требований к качеству и функциональности мобильных приложений. Современные пользователи ожидают от приложений не только привлекательного интерфейса, но и стабильной работы, быстрого отклика, поддержки офлайн-режима и корректной синхронизации данных.

В условиях постоянно растущей конкуренции на рынке мобильных приложений разработчики вынуждены применять современные архитектурные подходы и использовать актуальные инструменты разработки. Одной из наиболее эффективных архитектурных моделей для Android-приложений является MVVM (Model-View-ViewModel), которая обеспечивает четкое разделение ответственности между компонентами, упрощает тестирование и поддержку кода.

Данный курсовой проект посвящен проектированию и реализации мобильного клиента для обмена сообщениями с использованием архитектуры MVVM и современного стека технологий Android. В процессе разработки были применены ключевые библиотеки и компоненты Android Jetpack: Room для локального хранения данных, Retrofit для работы с сетевыми API, LiveData и ViewModel для управления состоянием, Navigation Component для навигации между экранами, а также WorkManager для фоновой синхронизации и уведомлений.

Актуальность данной работы обусловлена необходимостью изучения и практического применения современных принципов разработки Android-приложений. Полученные в ходе выполнения курсового проекта навыки позволяют создавать масштабируемые, производительные и удобные в сопровождении мобильные приложения, соответствующие текущим стандартам индустрии.

1 Цель и задачи курсового проекта

Целью курсового проекта является разработка Android-приложение, демонстрирующее основные принципы Android-разработки, включая навигацию, архитектуру MVVM, работу с состоянием, сетью, локальной базой данных и фоновыми задачами.

Задачами курсового проекта являются:

- освоение навыков по созданию приложения с Activity, Fragment и Navigation Component;
- реализация управления состоянием и архитектуру MVVM с использованием ViewModel и LiveData;
- приобретение навыков работы с сетевыми API через Retrofit и корутины;

- реализация локальное хранение данных с помощью Room и офлайн-доступ;
- улучшение пользовательского интерфейса с применением Material Design.
- настройка фоновых задач и уведомлений через WorkManager.

2 Этапы курсового проекта

2.1 Лабораторная работа №1

2.1.1 Цель лабораторной работы

Разработать простое Android-приложение с использованием архитектуры на основе Activity и Fragment, а также встроенной навигации.

2.1.2 Стэк

Был задан стэк: Java/Kotlin.

2.1.3 Функциональные требования

Функциональные требования:

1. главная активность приложения как точка входа;
2. навигация через Bottom Navigation;
3. вкладки: экран новостной ленты, экран профиля, экран настроек;
4. навигация реализована с использованием NavController и Navigation Graph;
5. отслеживание жизненного цикла компонентов с логированием.

2.1.4 Нефункциональные требования

Нефункциональные требования:

1. структурированный и расширяемый код;
2. приложение запускается без ошибок;
3. понятный и удобный интерфейс;
4. настроен .gitignore, исключающий лишние файлы;
5. APK приложен для проверки работы.

2.1.5 Выполнение лабораторной работы

В рамках лабораторной работы №1 было разработано простое Android-приложение, использующее архитектуру на основе Activity и Fragment, а также встроенную навигацию с помощью Navigation Component и Bottom Navigation.

Приложение содержит одну главную активность (MainActivity) и три фрагмента:

- экран новостной ленты (FeedFragment);

- экран профиля (ProfileFragment);
- экран настроек (SettingsFragment).

Для отслеживания жизненного цикла компонентов было реализовано логирование через вспомогательный класс Logger.

2.1.5.1 Главная активность - MainActivity

MainActivity является точкой входа в приложение. В ней настраивается NavHostFragment, извлекается NavController и связывается с BottomNavigationView:

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Logger.log("MainActivity created")

        setContentView(R.layout.activity_main)

        val navHostFragment = supportFragmentManager
            .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
        val navController = navHostFragment.navController

        val bottomNav =
            findViewById<BottomNavigationView>(R.id.bottom_navigation)
        bottomNav.setupWithNavController(navController)
    }

    override fun onDestroy() {
        super.onDestroy()
        Logger.log("MainActivity destroyed")
    }
}
```

}

2.1.5.2 Экран новостной ленты - FeedFragment

FeedFragment реализует экран отображения новостной ленты и демонстрирует полный набор основных методов жизненного цикла фрагмента. В методах `onAttach`, `onCreate`, `onCreateView`, `onViewCreated`, `onDestroyView`, `onDestroy` и `onDetach` вызывается вспомогательный метод логирования. В `onCreateView` выполняется инфлейт макета `fragment_feed`. Такое решение позволяет наглядно проследить, в каком порядке вызываются методы жизненного цикла при первом открытии экрана и при возврате к нему после навигации.

2.1.5.3 Экран профиля - ProfileFragment

ProfileFragment отвечает за отображение экрана профиля пользователя и по своей структуре полностью аналогичен FeedFragment. В нем переопределены те же методы жизненного цикла с добавлением сообщений в лог. В методе `onCreateView` инфлейтится макет `fragment_profile`. Благодаря одинаковой структуре с другими фрагментами достигается единообразие кода, что упрощает его поддержку и расширение. Логирование позволяет сравнить жизненный цикл различных экранов при навигации.

2.1.5.4 Экран настроек - SettingsFragment

SettingsFragment предназначен для отображения экрана настроек приложения. Его реализация повторяет общий шаблон, использованный в других фрагментах: переопределены ключевые методы жизненного цикла и добавлены сообщения в лог. В `onCreateView` происходит инфлейт макета `fragment_settings`. Такой подход обеспечивает единый стиль реализации экранов и позволяет отслеживать поведение фрагмента при переходах между пунктами нижнего меню навигации.

2.1.5.5 Вспомогательный класс логирования - Logger

Для централизованного логирования был создан объект `Logger`, инкапсулирующий работу с методом `Log.d`. В нем задан единый тег (`LifecycleLogger`), что упрощает фильтрацию сообщений в `Logcat`. Метод `log` принимает строковое сообщение и выводит его в лог. Использование отдельного класса для логирования повышает читаемость кода, снижает дублирование и позволяет при необходимости легко изменить способ вывода диагностической информации.

2.1.5.6 Разметка и XML-ресурсы приложения

Разметка и XML-ресурсы приложения были реализованы для обеспечения корректной работы навигации и формирования базового пользовательского интерфейса. Главная разметка активности построена на основе `ConstraintLayout` и включает контейнер для навигационного хоста

(FragmentManager) и нижнюю панель навигации (BottomNavigationView). Контейнер выступает в роли NavHostFragment, связан с графом навигации и занимает всё доступное пространство экрана над нижним меню, что обеспечивает корректную смену фрагментов при навигации. Листинг контейнера навигационного хоста приведен ниже:

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/nav_host_fragment"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="0dp"
    android:layout_height="0dp"
    app:defaultNavHost="true"
    app:layout_constraintBottom_toTopOf="@id/bottom_navigation"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:navGraph="@navigation/nav_graph" />
```

Нижняя навигационная панель закреплена у нижней границы экрана, использует меню bottom_nav_menu.xml и отображает подписи под иконками вкладок, обеспечивая удобное переключение между экранами.

Экран новостной ленты реализован с использованием FrameLayout, внутри которого размещен один TextView. Такой минималистичный макет используется в качестве заглушки для демонстрации экрана, а текст центрируется по горизонтали и вертикали.

Экран профиля пользователя построен на основе LinearLayout с вертикальной ориентацией. Внутри размещены изображение профиля (ImageView) и имя пользователя (TextView), элементы центрируются по экрану и имеют отступы для улучшения визуального восприятия. В качестве заглушки используется стандартная иконка приложения.

Экран настроек реализован в минималистичном стиле и содержит один элемент Switch, предназначенный для демонстрации переключателя тёмной темы без реализации логики:

```
<Switch
```

```
    android:id="@+id/themeSwitch"
    android:layout_width="wrap_content"
```

```
android:layout_height="wrap_content"

android:text="Тёмная тема"

android:layout_gravity="center"

tools:ignore="UseSwitchCompatOrMaterialXml" />
```

Меню нижней навигации содержит три пункта, соответствующие экранам новостей, профиля и настроек. Каждый пункт имеет уникальный идентификатор, заголовок и стандартную иконку Android, которые используются в графе навигации для связывания меню с фрагментами:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item

        android:id="@+id/feedFragment"

        android:title="Новости"

        android:icon="@android:drawable/ic_menu_view" />

    <item

        android:id="@+id/profileFragment"

        android:title="Профиль"

        android:icon="@android:drawable/ic_menu_myplaces" />

    <item

        android:id="@+id/settingsFragment"

        android:title="Настройки"

        android:icon="@android:drawable/ic_menu_manage" />

</menu>
```

Навигация между экранами описана в XML-графе навигации, в котором задан стартовый экран (FeedFragment) и определены все фрагменты с указанием их классов и макетов. Этот граф используется объектом NavController для управления переходами между вкладками.

Для корректной работы темы и базового оформления приложения были настроены стандартные ресурсы. В файле colors.xml задан набор базовых цветов, используемых темой приложения.

В файле strings.xml задано имя приложения, отображаемое в системе:

```
<resources>
```

```
    <string name="app_name">labs_dk_lab1</string>
```

```
</resources>
```

Тема приложения определена в themes.xml и основана на светлой теме Material Design без стандартного ActionBar.

В процессе разработки XML-ресурсов интерфейса и навигации были реализованы следующие элементы: создана основная разметка с использованием NavHostFragment и BottomNavigationView, определены макеты для трех экранов, описано меню нижней навигации и соответствующий граф навигации. Кроме того, были настроены ресурсы цветов, строк и тема приложения. Эти меры обеспечили корректную работу навигационных функций и формирование базового пользовательского интерфейса приложения.

2.2 Лабораторная работа №2

2.2.1 Цель лабораторной работы

Реализовать управление состоянием приложения и архитектуру MVVM для хранения и обновления пользовательских данных в экранах

2.2.2 Функциональные требования

Функциональные требования:

1. Добавление ViewModel для хранения состояния профиля и настроек;
2. Экран «Профиль» с отображением имени и статуса пользователя через EditText;
3. Экран «Настройки» с переключателем темы (светлая/темная);
4. Реактивное обновление UI через LiveData;
5. Сохранение данных при повороте экрана;
6. Логирование жизненного цикла ViewModel.

2.2.3 Нефункциональные требования

Нефункциональные требования:

1. Применение принципов разделения ответственности между UI и ViewModel;
2. Обеспечение устойчивой работы при изменении конфигурации.

2.2.4 Выполнение лабораторной работы

В рамках второй лабораторной работы в приложение была внедрена архитектура MVVM (Model-View-ViewModel) для управления состоянием

экранов «Профиль» и «Настройки». Состояние пользовательских данных (имя, статус, тема оформления) вынесено из UI-слоя во ViewModel, что обеспечило:

- сохранение данных при повороте экрана;
- реактивное обновление интерфейса через LiveData;
- разделение ответственности между UI и бизнес-логикой.

2.2.4.1 ViewModel для хранения состояния

Разработан класс AppViewModel, предназначенный для хранения данных о профиле и настройках:

- Имя пользователя (userName);
- Статус пользователя (userStatus);
- Выбранная тема (isDarkTheme).

Для динамического обновления пользовательского интерфейса применяются LiveData и MutableLiveData.

2.2.4.2 Экран профиля - ProfileFragment

Экран профиля был модернизирован с использованием ViewModel и ViewBinding. Основные нововведения включают:

- интеграцию AppViewModel через метод activityViewModels();
- подписку на LiveData;
- обратную передачу изменений из EditText в ViewModel.

Интерфейс пользователя откликается на изменения состояния. Ввод пользователя синхронизируется с ViewModel. При повороте экрана данные сохраняются.

В результате UI реагирует на изменения состояния в реальном времени, пользовательский ввод автоматически синхронизируется с ViewModel, и данные сохраняются при повороте экрана, обеспечивая непрерывность работы приложения.

2.2.4.3 Экран настроек - SettingsFragment

Экран настроек был доработан для хранения состояния темы во ViewModel:

```
private val viewModel: AppViewModel by activityViewModels()
```

```
viewModel.isDarkTheme.observe(viewLifecycleOwner) {
```

```
    binding.themeSwitch.isChecked = it
```

```
}
```

```
binding.themeSwitch.setOnCheckedChangeListener { _, checked ->
    viewModel.setTheme(checked)
}
```

Состояние переключателя сохраняется в ViewModel. Пользовательский интерфейс обновляется автоматически при изменении LiveData. Это обеспечивает реактивное обновление интерфейса.

2.2.4.4 Разделение ответственности (MVVM)

В архитектуре MVVM UI (Fragment) отвечает за визуализацию данных и обработку пользовательских взаимодействий, в то время как ViewModel (AppViewModel) управляет состоянием и бизнес-логикой, не связываясь с View. Такой подход улучшает масштабируемость кода и облегчает его поддержку.

2.2.4.5 Логирование жизненного цикла ViewModel

Введено логирование для отслеживания следующих событий:

- Создание ViewModel;
- Изменение данных;
- Очистка ViewModel.

Листинг кода:

```
init {
    Logger.log("AppViewModel created")
}
```

```
override fun onCleared() {
    super.onCleared()
    Logger.log("AppViewModel cleared")
}
```

Это позволяет отследить, когда ViewModel создается и уничтожается.

2.2.4.6 Изменения в XML-ресурсах

Разметка экрана профиля была переработана в соответствии с архитектурным подходом MVVM. Вместо статичного отображения имени пользователя теперь используются два поля ввода EditText: editName и editStatus, предназначенные для ввода имени и статуса пользователя соответственно. Эти элементы интерфейса связываются с ViewModel через LiveData, что обеспечивает двустороннюю синхронизацию данных между пользовательским интерфейсом и логикой приложения. Структура макета реализована в виде вертикального LinearLayout с внутренними отступами (padding), минимальной высотой для полей ввода и расстоянием между элементами для улучшения читаемости и удобства взаимодействия. Главное изменение заключается в том, что экран профиля стал редактируемым и реактивным: введенные пользователем данные автоматически отражаются в ViewModel и могут быть восстановлены при пересоздании фрагмента, например, при повороте экрана.

Экран настроек также был модифицирован для соответствия архитектуре MVVM. Вместо одного простого переключателя темной темы теперь используется Switch (themeSwitch), привязанный к LiveData во ViewModel. Это позволяет сохранять состояние переключателя при повороте экрана и других изменениях конфигурации, обеспечивая согласованность пользовательского интерфейса с текущими настройками приложения. Разметка экрана реализована в виде вертикального LinearLayout с внутренними отступами (padding) и минимальной высотой для элемента переключения, что делает интерфейс более аккуратным и удобным. Главное изменение состоит в том, что состояние переключателя теперь хранится во ViewModel, а не в самом фрагменте, что повышает устойчивость приложения к пересозданию компонентов и упрощает управление состоянием.

2.3 Лабораторная работа №3

2.3.1 Цель лабораторной работы

Добавить загрузку списка сообщений из API и реализовать сохранение данных в локальную базу Room для офлайн-доступа

2.3.2 Функциональные требования

Функциональные требования:

1. создание MessageRepository для получения данных из публичного API;
2. реализация Retrofit-клиента с использованием корутин;
3. отображение списка сообщений на вкладке «Лента» с помощью RecyclerView;
4. сохранение данных в локальную базу Room;
5. загрузка сообщений из базы при отсутствии сети;
6. кнопка «Обновить» для ручного обновления данных.

2.3.3 Нефункциональные требования

Нефункциональные требования:

1. сетевые операции только через корутины;
2. работа с базой через DAO и Entity;
3. репозиторий - единая точка взаимодействия ViewModel с данными;
4. обеспечение офлайн-доступа к данным.

2.3.4 Выполнение лабораторной работы

На этом этапе разработки приложения была реализована полноценная цепочка получения, хранения и отображения сообщений:

API → Repository → Room → ViewModel → UI (RecyclerView)

В приложении были внедрены следующие улучшения:

- реализована загрузка сообщений через публичное REST API;
- внедрена система локального кэширования с использованием Room;
- добавлена возможность работы в офлайн-режиме;
- реализована функция ручного обновления данных;
- создан единый репозиторий для доступа к данным;
- все сетевые операции выполняются исключительно через корутины.

2.3.4.1 Изменения в слое данных

В рамках третьей лабораторной работы был существенно доработан слой данных приложения. Добавлена поддержка сетевых запросов, локального кэширования и репозитория как единой точки доступа к данным.

Для получения сообщений из сети реализован интерфейс MessengerApi, использующий библиотеку Retrofit. В нём определён метод getMessages(), помеченный как suspend, что обеспечивает выполнение сетевых операций через корутины. Базовый URL API вынесен в константу BASE_URL.

Для создания и настройки Retrofit-клиента добавлен класс RetrofitClient, реализованный в виде singleton-объекта. В нём настроен OkHttpClient с таймаутами и конвертером GsonConverterFactory. Клиент предоставляет готовый экземпляр MessengerApi для выполнения сетевых запросов.

Модель Message была расширена и адаптирована под использование Room. Класс помечен аннотацией @Entity, добавлен первичный ключ @PrimaryKey, а также поле timestamp, которое используется для сортировки

сообщений и отображения времени. Аннотации `@SerializedName` обеспечивают корректное сопоставление JSON-полей с полями модели.

Для работы с локальной базой данных реализован DAO-интерфейс `MessageDao`, содержащий методы для получения всех сообщений в виде `LiveData`, синхронного получения списка сообщений, вставки данных с заменой конфликтов и очистки таблицы. Это позволяет как подписываться на изменения данных в базе, так и загружать кэшированные данные при отсутствии сети.

Создана база данных `MessengerDatabase` на основе `Room`. Она содержит сущность `Message` и предоставляет доступ к `MessageDao`. База реализована как `singleton`, что гарантирует использование единственного экземпляра во всём приложении.

Ключевым элементом слоя данных стал класс `MessageRepository`. Он инкапсулирует логику работы с сетью и локальной базой данных и выступает единой точкой взаимодействия для `ViewModel`. Репозиторий определяет, откуда загружать данные: из API или из локального кэша, сохраняет результаты сетевых запросов в `Room`, а также проверяет наличие сетевого подключения. Все операции с сетью и базой выполняются в `Dispatchers.IO`. Для обработки ошибок используется тип `Result`.

Таким образом, слой данных был переработан таким образом, чтобы обеспечить асинхронную загрузку сообщений из сети, их локальное кэширование и поддержку офлайн-доступа при сохранении единой архитектурной точки входа к данным через репозиторий.

2.3.4.2 Изменения в UI-слое

Для получения сообщений из сети реализован интерфейс `MessengerApi`, использующий библиотеку `Retrofit`. В нём определён метод `getMessages()`, помеченный как `suspend`, что обеспечивает выполнение сетевых операций через корутины. Базовый URL API вынесен в константу `BASE_URL`.

```

interface MessengerApi {

    1 Usage
    @GET( value = "posts")
    suspend fun getMessages(): List<Message>

    1 Usage
    companion object {
        1 Usage
        const val BASE_URL = "https://jsonplaceholder.typicode.com/"
    }
}

```

Рисунок 1 - код MessengerApi.kt

Для создания и настройки Retrofit-клиента добавлен класс `RetrofitClient`, реализованный в виде singleton-объекта. В нём настроен `OkHttpClient` с таймаутами и конвертером `GsonConverterFactory`. Клиент предоставляет готовый экземпляр `MessengerApi` для выполнения сетевых запросов.

```

object RetrofitClient {

    1 Usage
    private val loggingInterceptor = HttpLoggingInterceptor().apply {
        level = HttpLoggingInterceptor.Level.BODY
    }

    1 Usage
    private val okHttpClient = OkHttpClient.Builder()
        .addInterceptor(loggingInterceptor)
        .connectTimeout( timeout = 30, unit = TimeUnit.SECONDS)
        .readTimeout( timeout = 30, unit = TimeUnit.SECONDS)
        .writeTimeout( timeout = 30, unit = TimeUnit.SECONDS)
        .build()

    1 Usage
    private val retrofit = Retrofit.Builder()
        .baseUrl( baseUrl = MessengerApi.BASE_URL)
        .client(okHttpClient)
        .addConverterFactory( factory = GsonConverterFactory.create())
        .build()

    val api: MessengerApi = retrofit.create(MessengerApi::class.java)
}

```

Рисунок 2 - код RetrofitClient.kt

2.3.4.3 Локальная база данных Room

Модель Message была расширена и адаптирована под использование Room. Класс помечен аннотацией @Entity, добавлен первичный ключ @PrimaryKey, а также поле timestamp, которое используется для сортировки сообщений и отображения времени. Аннотации @SerializedName обеспечивают корректное сопоставление JSON-полей с полями модели:

```
@Entity(tableName = "messages")

data class Message(

    @PrimaryKey

    @SerializedName("id") val id: Int,

    ...

    val timestamp: Long = System.currentTimeMillis()

)
```

Для работы с локальной базой данных реализован DAO-интерфейс MessageDao, содержащий методы для получения всех сообщений в виде LiveData, синхронного получения списка сообщений, вставки данных с заменой конфликтов и очистки таблицы. Это позволяет как подписываться на изменения данных в базе, так и загружать кэшированные данные при отсутствии сети.

```

@Dao
interface MessageDao {

    1 Usage 1 Implementation
    @Query(value = "SELECT * FROM messages ORDER BY timestamp DESC")
    fun getAllMessages(): LiveData<List<Message>>

    1 Usage 1 Implementation
    @Query(value = "SELECT * FROM messages ORDER BY timestamp DESC")
    suspend fun getAllMessagesSync(): List<Message>

    1 Usage 1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertMessages(messages: List<Message>)

    1 Usage 1 Implementation
    @Query(value = "DELETE FROM messages")
    suspend fun deleteAllMessages()

    1 Implementation
    @Query(value = "SELECT COUNT(*) FROM messages")
    suspend fun getMessageCount(): Int
}

```

Рисунок 3 - код MessageDao.kt

Создана база данных MessengerDatabase на основе Room. Она содержит сущность Message и предоставляет доступ к MessageDao. База реализована как singleton, что гарантирует использование единственного экземпляра во всём приложении.

```

@Database(entities = [Message::class], version = 1, exportSchema = false)
abstract class MessengerDatabase : RoomDatabase() {

    1 Usage 1 Implementation
    abstract fun messageDao(): MessageDao
}

```

Рисунок 4 - отрывок кода MessengerDatabase.kt

2.3.4.4 Новый файл - MessageRepository.kt

Ключевым элементом слоя данных стал класс MessageRepository. Он инкапсулирует логику работы с сетью и локальной базой данных и выступает единой точкой взаимодействия для ViewModel. Репозиторий определяет, откуда загружать данные: из API или из локального кэша, сохраняет

результаты сетевых запросов в Room, а также проверяет наличие сетевого подключения. Все операции с сетью и базой выполняются в Dispatchers.IO. Для обработки ошибок используется тип Result:

```
class MessageRepository(private val context: Context) {  
    val allMessages: LiveData<List<Message>> =  
        messageDao.getAllMessages()  
  
    suspend fun refreshMessages(): Result<Unit> { ... }  
  
    suspend fun loadCachedMessages(): List<Message> { ... }  
}
```

2.3.4.5 Изменения в UI-слое

В третьей лабораторной работе UI-слой приложения был существенно доработан для поддержки динамических данных, получаемых из сети и локальной базы, а также для реализации ручного обновления, состояний загрузки и обработки ошибок. Если в предыдущих лабораторных экраны в основном отображали статические данные, то теперь вкладка «Лента» стала полноценным реактивным экраном, связанным с ViewModel и репозиторием.

Основные изменения были внесены во фрагмент FeedFragment, который теперь отвечает за отображение списка сообщений. В разметку экрана был добавлен RecyclerView, а также элементы для индикации загрузки (ProgressBar), пустого состояния и кнопка «Обновить» для ручного запроса данных. Во фрагменте реализована привязка адаптера MessageAdapter и настроены наблюдатели за LiveData, предоставляемыми FeedViewModel. При изменении списка сообщений UI автоматически обновляется, а при пустом списке отображается соответствующее сообщение:

```
viewModel.messages.observe(viewLifecycleOwner) { messages ->  
    adapter.submitList(messages)  
  
    if (messages.isEmpty()) {  
        binding.textEmptyState.visibility = View.VISIBLE  
        binding.recyclerViewMessages.visibility = View.GONE  
    } else {
```

```

        binding.textEmptyState.visibility = View.GONE

        binding.recyclerViewMessages.visibility = View.VISIBLE
    }
}

```

Для ручного обновления данных добавлена кнопка «Обновить», нажатие на которую инициирует загрузку сообщений через ViewModel:

```

binding.buttonRefresh.setOnClickListener {
    viewModel.loadMessages()
}

```

FeedViewModel был переработан для взаимодействия с репозиторием и поддержки состояний загрузки и ошибок. В него добавлены LiveData<List<Message>> для списка сообщений, LiveData<Boolean> для состояния загрузки и LiveData<String?> для сообщений об ошибке. Все операции загрузки выполняются в viewModelScope с использованием корутин, что позволяет безопасно выполнять фоновые операции и переживать изменения конфигурации экрана. При запуске загрузки ViewModel обновляет состояние загрузки, обращается к репозиторию и обрабатывает возможные ошибки:

```

fun loadMessages() {
    viewModelScope.launch {
        _isLoading.value = true
        _errorMessage.value = null

        val result = repository.refreshMessages()

        result.onFailure {
            _errorMessage.value = "Не удалось загрузить сообщения.
Показаны данные из кэша."
        }

        _isLoading.value = false
    }
}

```

```
}
```

```
}
```

Для отображения списка сообщений реализован адаптер `MessageAdapter`, основанный на `ListAdapter` и `DiffUtil`, что обеспечивает эффективное обновление элементов списка при изменении данных. В адаптере используется `ViewHolder` с биндингом `ItemMessageBinding`, а также реализовано форматирование времени сообщения (`timestamp`) для отображения даты и времени в читаемом виде. Привязка данных к элементам списка выполняется следующим образом:

```
fun bind(message: Message) {  
    binding.apply {  
        textTitle.text = message.title  
        textBody.text = message.body  
        textUser.text = "User #${message.userId}"  
        textTimestamp.text = dateFormat.format(Date(message.timestamp))  
    }  
}
```

Вкладки «Профиль» и «Настройки» сохранили общую структуру, но были адаптированы под обновленную архитектуру MVVM. Для обоих экранов используются соответствующие `ViewModel` (`ProfileViewModel` и `SettingsViewModel`), а все данные экрана теперь хранятся в `LiveData`. UI реагирует на изменения состояния через наблюдателей, что делает экраны реактивными и устойчивыми к пересозданию фрагментов, например, при повороте экрана.

В результате доработки UI-слоя был добавлен экран со списком сообщений на основе `RecyclerView`, реализована кнопка ручного обновления данных, добавлены состояния загрузки, пустого списка и ошибки, а также выполнен переход на реактивную модель с использованием `LiveData` и `ViewModel`. Вся логика получения данных была вынесена из фрагментов в `ViewModel` и репозиторий, что обеспечило более чистое разделение ответственности между UI, `ViewModel` и слоем данных, а также поддержку сетевой загрузки и офлайн-доступа.

2.3.4.6 Роли компонентов

В архитектуре приложения UI-слой, представленный фрагментами, отвечает исключительно за отображение данных и обработку действий пользователя. Фрагменты не содержат бизнес-логики и не взаимодействуют

напрямую ни с сетью, ни с базой данных. Они подписываются на LiveData, предоставляемые ViewModel, и автоматически обновляют интерфейс при изменении данных. Любые действия пользователя, такие как нажатие кнопки «Обновить», передаются во ViewModel, что обеспечивает четкое разделение ответственности и упрощает сопровождение и тестирование кода.

ViewModel выполняет роль посредника между пользовательским интерфейсом и слоем данных. Она хранит состояние экрана, управляет логикой, связанной с отображением сообщений, и запускает корутины через viewModelScope для выполнения фоновых операций. ViewModel вызывает методы репозитория, обрабатывает результаты загрузки, состояние ошибок и индикаторы прогресса, при этом не зная, откуда именно поступают данные - из сети или из локальной базы. Такое разделение позволяет изолировать UI от деталей реализации источников данных и делает код устойчивым к изменениям конфигурации, например, при повороте экрана.

Репозиторий является единой точкой доступа к данным в приложении и инкапсулирует всю логику работы с источниками информации. Именно он решает, нужно ли загружать сообщения из сети через Retrofit или использовать локальный кэш Room, а также отвечает за сохранение сетевых данных в базу, обработку офлайн-режима и проверку доступности сети. Репозиторий скрывает от остальных слоев детали работы с API и БД и предоставляет ViewModel простой и согласованный интерфейс, что упрощает развитие приложения и добавление новых источников данных.

Room используется как локальное хранилище данных и основной источник информации для оффлайн-режима. Сущность Entity описывает структуру таблицы сообщений, Dao определяет методы доступа к данным, а класс базы данных объединяет их в единый объект. Room возвращает данные в виде LiveData, благодаря чему пользовательский интерфейс автоматически обновляется при изменениях в базе. Это позволяет приложению корректно работать без подключения к интернету и обеспечивает сохранность данных между запусками.

Retrofit отвечает за получение данных из сети и взаимодействие с REST API. Все сетевые запросы реализованы через suspend-функции и выполняются в корутинах, что исключает блокировку основного потока. Retrofit используется только внутри репозитория и недоступен напрямую ни ViewModel, ни UI-слою, что обеспечивает строгую изоляцию сетевого слоя и упрощает возможную замену API или изменение логики загрузки данных в будущем.

В совокупности такая архитектура MVVM с использованием Repository, Room и Retrofit обеспечивает чёткое разделение ответственности между слоями, упрощает сопровождение кода, делает приложение устойчивым к изменениям конфигурации и позволяет масштабировать функциональность без переписывания существующих компонентов.

2.3.4.7 Изменения в XML-ресурсах

По сравнению со второй лабораторной работой, в третьей версии приложения были внесены как архитектурные, так и функциональные изменения, затрагивающие пользовательский интерфейс, навигацию и структуру экранов. Основная цель этих доработок заключалась в переходе от статического интерфейса к динамическому, поддерживающему загрузку данных из сети, офлайн-доступ и реактивное обновление UI в соответствии с архитектурой MVVM.

Разметка главного экрана (activity_main.xml) в целом сохранила прежнюю концепцию с использованием `FragmentContainerView` и `BottomNavigationView`, однако была приведена к более консистентному Material-стилю. В частности, фон нижней навигации был изменён с `?android:attr/windowBackground` на `?attr/colorSurface`, что обеспечивает корректную интеграцию с темой Material Design. Кроме того, разметка была упрощена за счет удаления `tools:*`-атрибутов, а навигация синхронизирована с обновленным графом навигации. Логика работы `NavHostFragment` при этом осталась без изменений.

Экран «Лента» был полностью переработан. Если во второй лабораторной работе он представлял собой `FrameLayout` с одним статическим `TextView`, то в третьей версии он реализован на основе `ConstraintLayout` и стал динамическим. В разметку были добавлены заголовок экрана, кнопка «Обновить», `ProgressBar` для индикации загрузки, `TextView` для отображения пустого состояния и `RecyclerView` для списка сообщений. Это позволило экрану поддерживать отображение данных, полученных из сети, работу с офлайн-кэшем и ручное обновление контента.

Для поддержки списка сообщений была добавлена новая разметка элемента списка (item_message.xml). Ранее в приложении отсутствовал список и отдельный layout для элементов. В третьей лабораторной работе был реализован `MaterialCardView`, внутри которого отображаются заголовок сообщения, тело сообщения, идентификатор пользователя (userId) и временная метка (timestamp). Это обеспечило полноценное отображение элементов списка с использованием компонентов Material Design и улучшило визуальное восприятие контента.

Экран «Профиль» также претерпел существенные изменения. Если во второй лабораторной работе он представлял собой простой `LinearLayout` с двумя `EditText`, то в третьей версии экран был обернут в `ScrollView` и структурирован на логические секции. В разметку были добавлены заголовок экрана, карточка профиля (`MaterialCardView`) с отображением имени и статуса пользователя, отдельная секция редактирования профиля, `TextInputLayout` и `TextInputEditText`, а также кнопки сохранения имени и статуса. В результате экран стал более интерактивным, визуально аккуратным и лучше подготовленным к расширению функциональности.

Изменения коснулись и экрана «Настройки», а также меню нижней навигации. В меню были обновлены иконки пунктов: вместо `ic_menu_view` и `ic_menu_manage` теперь используются `ic_dialog_email` и `ic_menu_preferences`, а название первой вкладки изменено с «Новости» на «Лента». Эти правки позволили синхронизировать семантику меню с новым назначением экрана сообщений и улучшить визуальное соответствие иконок содержимому вкладок.

Граф навигации был приведён в соответствие с новой архитектурой и пакетной структурой проекта. Были удалены `tools:layout`, использовавшиеся ранее для предпросмотра, и обновлены пути к классам фрагментов (с `com.m.labs_dk_lab1.ui.*` на `com.m.cursproject.ui.*`). При этом стартовый экран остался прежним - вкладка «Лента», что сохранило привычную логику запуска приложения.

В итоге, по сравнению с предыдущей лабораторной работой, экран «Лента» был полностью переработан из статического в динамический, добавлены `RecyclerView`, карточки сообщений и пустое состояние, улучшен UI экрана «Профиль» с использованием Material-компонентов, обновлены иконки и подписи в нижней навигации, упрощена разметка за счёт удаления `tools:*`-атрибутов и обновлены пакеты и навигационный граф под новую архитектуру MVVM. Эти изменения обеспечили визуальное улучшение интерфейса, поддержку сетевых данных и офлайн-доступа, а также подготовили UI-слой к масштабируемому развитию.

2.4 Лабораторная работа №4

2.4.1 Цель лабораторной работы

Улучшить пользовательский интерфейс, добавить уведомления и фоновую синхронизацию сообщений.

2.4.2 Функциональные требования

Функциональные требования:

1. реализован кастомный `RecyclerView.Adapter` для отображения аватарки, имени и текста сообщения;
2. применены компоненты Material Design: `CardView`, `FloatingActionButton`, `AppBarLayout`;
3. добавлена возможность «лайкать» сообщение с изменением иконки при клике;
4. настроен `WorkManager` для периодического обновления списка сообщений из сети;
5. при успешной синхронизации показываются `Notification` с текстом «Новые данные получены»;
6. запрос разрешений при запуске приложения (уведомления, контакты и т.п.);

7. приложение корректно реагирует на системные события (онлайн/офлайн);

8. код оформлен согласно принципам MVVM.

2.4.3 Нефункциональные требования

Нефункциональные требования:

1. уведомления должны работать в фоне;
2. UI должен быть удобным и современным;
3. код структурирован и поддерживает расширение функционала;
4. обеспечение стабильной работы при изменении состояния сети.

2.4.4 Выполнение лабораторной работы

В рамках четвертой лабораторной работы пользовательский интерфейс и архитектура приложения были расширены для поддержки интерактивных элементов, фоновой синхронизации и уведомлений. Все изменения выполнены в соответствии с принципами MVVM.

2.4.4.1 Кастомный список сообщений

Для отображения сообщений используется `MessageAdapter`, который поддерживает визуализацию изображений пользователей, их имён и текстового содержимого сообщений.

Ключевыми нововведениями стали:

- внедрение `ListAdapter` в комбинации с `DiffUtil`, что обеспечивает оперативное обновление списка;
- использование библиотеки `Glide` для загрузки изображений пользователей;
- отображение в списке следующих полей: `title`, `body`, `userName` и `timestamp`;
- реализация функции лайка, которая изменяет значок и цвет кнопки.

```

class MessageAdapter(
    class MessageViewHolder(
        fun bind(message: Message) {
            binding.apply {

                if (message.avatarUrl.startsWith( prefix = "drawable://")) {
                    val drawableName = message.avatarUrl.removePrefix( prefix = "drawable://")
                    val drawableId = imageAvatar.context.resources.getIdentifier(
                        drawableName,
                        defType = "drawable",
                        defPackage = imageAvatar.context.packageName
                    )
                    if (drawableId != 0) {
                        val drawable = ContextCompat.getDrawable( context = imageAvatar.context, drawableId)
                        drawable?.let {
                            Glide.with( context = imageAvatar.context)
                                .load( drawable = it)
                                .circleCrop()
                                .into( view = imageAvatar)
                        } ?: imageAvatar.setImageResource(R.drawable.ic_avatar_placeholder)
                    } else {
                        imageAvatar.setImageResource(R.drawable.ic_avatar_placeholder)
                    }
                } else {
                    Glide.with( context = imageAvatar.context)
                        .load( string = message.avatarUrl)
                        .placeholder( resourceId = R.drawable.ic_avatar_placeholder)
                        .error( resourceId = R.drawable.ic_avatar_placeholder)
                        .circleCrop()
                        .into( view = imageAvatar)
                }
            }
        }
    )
)

```

Рисунок 5 - Улучшенный MessageAdapter.kt

2.4.4.2 Применение компонентов Material Design

В UI-слое использованы стандартные Material-компоненты для улучшения внешнего вида и UX. Было добавлено:

- CardView - для карточек сообщений;
- FloatingActionButton - для ручного обновления ленты;
- AppBarLayout - для верхней панели приложения.

2.4.4.3 Поддержка лайков сообщений

Поддержка лайков сообщений реализована за счёт расширения модели данных и обновления логики репозитория и UI. В модель Message добавлено булево поле isLiked, которое хранит текущее состояние лайка для каждого сообщения. В слое данных (MessageRepository) реализован метод toggleLike, переключающий это состояние и уведомляющий подписчиков об изменениях. На уровне интерфейса кнопка лайка (ImageButton в item_message.xml) изменяет свою иконку при клике, обеспечивая наглядную визуальную индикацию того, отмечено ли сообщение пользователем. Это позволяет синхронизировать состояние данных и отображение в UI, улучшая интерактивность и пользовательский опыт.

2.4.4.4 Фоновая синхронизация через WorkManager

Для автоматического обновления списка сообщений в приложении реализован фоновый воркер SyncWorker на основе WorkManager. Он запускается периодически и выполняет синхронизацию данных с сервером без участия пользователя. Воркер настроен на выполнение каждые 15 минут с учётом системных ограничений: наличие активного интернет-соединения и достаточный уровень заряда батареи. Перед началом синхронизации выполняется проверка доступности сети. При отсутствии интернета задача не завершается с ошибкой, а откладывается с помощью Result.retry(). В случае успешного обновления данных пользователю отображается системное уведомление с сообщением «Новые данные получены». Такой подход обеспечивает корректную работу приложения в фоне и устойчивость к временным проблемам с сетью.

```
class SyncWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker( appContext = context, params ) {

    2 Usages
    private val repository = MessageRepository.getInstance(context)

    override suspend fun doWork(): Result {
        Log.d( tag = TAG, msg = "SyncWorker: Starting background sync")

        return try {
            if (!repository.isNetworkAvailable()) {
                Log.d( tag = TAG, msg = "SyncWorker: No network available")
                return Result.retry()
            }

            val result = repository.refreshMessages()

            if (result.isSuccess) {
                Log.d( tag = TAG, msg = "SyncWorker: Sync successful")
                showSuccessNotification()
                Result.success()
            } else {
                Log.e( tag = TAG, msg = "SyncWorker: Sync failed", tr = result.exceptionOrNull())
                Result.retry()
            }
        } catch (e: Exception) {
            Log.e( tag = TAG, msg = "SyncWorker: Exception during sync", tr = e)
            Result.failure()
        }
    }
}
```

Рисунок 6 - Первая часть кода SyncWorker.kt

2.4.4.5 Уведомления о новых данных

После успешной синхронизации сообщений в фоновом режиме пользователю отображается системное уведомление, информирующее о поступлении новых данных. Для корректной работы уведомлений в манифест приложения добавлено разрешение POST_NOTIFICATIONS, необходимое на устройствах с Android 13 и выше. В воркере SyncWorker

реализовано создание канала уведомлений для версий Android 8.0 и выше, что соответствует требованиям платформы. После этого формируется и отображается уведомление с текстом «Новые данные получены». Такой механизм позволяет оповещать пользователя о завершении фоновой синхронизации даже тогда, когда приложение не находится на экране.

```
1 Usage
private fun showSuccessNotification() {
    val notificationManager = applicationContext.getSystemService(name = Context.NOTIFICATION_SERVICE) as NotificationManager

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            id = CHANNEL_ID,
            name = "Синхронизация сообщений",
            importance = NotificationManager.IMPORTANCE_DEFAULT
        ).apply {
            description = "Уведомления о синхронизации данных"
        }
        notificationManager.createNotificationChannel(channel)
    }

    val notification = NotificationCompat.Builder(applicationContext, channelId = CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle("Новые данные")
        .setContentText("Новые данные получены")
        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
        .setAutoCancel(true)
        .build()

    notificationManager.notify(id = NOTIFICATION_ID, notification)

    Log.d(tag = TAG, msg = "SyncWorker: Notification shown")
}
```

Рисунок 7 - Первая часть кода SyncWorker.kt

2.4.4.6 Запрос разрешений и работа в онлайн/офлайн-режиме

При запуске приложения реализован запрос системных разрешений с использованием `ActivityResultContracts.RequestMultiplePermissions`. Пользователю предлагается предоставить доступ к уведомлениям (POST_NOTIFICATIONS) на устройствах с Android 13 и выше, а также к контактам (READ_CONTACTS). В зависимости от результата запроса приложение отображает Toast при успешном предоставлении разрешения и Snackbar в случае отказа, что обеспечивает наглядную обратную связь для пользователя.

Также приложение корректно обрабатывает работу в онлайн- и офлайн-режиме. В классе `MessageRepository` реализована проверка доступности сети через `ConnectivityManager`. При отсутствии интернет-соединения данные загружаются из локального кэша (Room), а фоновая синхронизация в `SyncWorker` не завершается с ошибкой, а откладывается с помощью `Result.retry()`. Такой подход позволяет сохранять работоспособность интерфейса и автоматически возобновлять синхронизацию при восстановлении сети.

2.4.4.7 Навигация и главный экран приложения

Главный экран приложения был доработан для поддержки современной навигации между основными разделами: лентой сообщений, профилем пользователя и настройками.

В MainActivity используется компонент Navigation Architecture Component, который обеспечивает централизованное управление переходами между фрагментами. Для этого в разметке активности размещён NavHostFragment, внутри которого отображаются все экраны приложения.

Нижнее меню (BottomNavigationView) связано с NavController, что позволяет автоматически синхронизировать выбранный пункт меню с текущим экраном и обрабатывать нажатия без ручной логики переходов. Таким образом, переходы между вкладками «Новости», «Профиль» и «Настройки» выполняются декларативно на основе графа навигации.

Вся логика навигации вынесена из фрагментов в главный Activity, что улучшает разделение ответственности: MainActivity отвечает за маршрутизацию между экранами, а каждый фрагмент - только за отображение UI и логику конкретного экрана.

Дополнительно в активности добавлено логирование основных событий жизненного цикла (onCreate, onStart, onResume, onPause, onStop, onDestroy), что упрощает отладку и позволяет отслеживать поведение приложения при смене состояний.

В результате внедрения Navigation Component и нижнего меню приложение получило единый механизм навигации, корректную работу кнопки «Назад», согласованность состояния UI с текущим экраном и более чистую архитектуру главного экрана.

2.4.4.8 Изменения в UI-слое

Пользовательский интерфейс приложения был существенно переработан по сравнению с третьей лабораторной работой. Основной упор сделан на соответствие принципам Material Design, повышение интерактивности и визуальной отзывчивости, а также на поддержку фоновой синхронизации данных. UI стал более структурированным, современным и логически разделенным на экраны.

Главный экран по-прежнему построен на ConstraintLayout, однако теперь используется связка Navigation Component и нижнего меню. В разметке размещён FragmentContainerView, который выступает единым контейнером для всех фрагментов, и BottomNavigationView для переключения между разделами.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:defaultNavHost="true"
        app:layout_constraintBottom_toTopOf="@id/bottom_navigation"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:navGraph="@navigation/nav_graph" />

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_navigation"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:background="?attr/colorSurface"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:menu="@menu/bottom_nav_menu" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Рисунок 8 - Код activity_main.xml

Это решение позволяет централизованно управлять навигацией, синхронизировать состояние нижнего меню с текущим экраном и избавиться от ручной логики переходов между фрагментами. В результате навигация стала более устойчивой и согласованной с жизненным циклом приложения.

Экран ленты сообщений был полностью переработан. Вместо простого ConstraintLayout теперь используется более сложная композиция элементов Material Design, обеспечивающая интерактивность и адаптацию под прокрутку.

Для поддержки жеста обновления реализован SwipeRefreshLayout:
<androidx.swiperefreshlayout.widget.SwipeRefreshLayout

```
android:id="@+id/swipe_refresh">
```

Также добавлена плавающая кнопка обновления:

```
<com.google.android.material.floatingactionbutton.FloatingActionButton  
    android:id="@+id/fab_refresh"  
    android:src="@drawable/ic_refresh" />
```

Для визуальной обратной связи используются индикатор загрузки и текст пустого состояния:

```
<ProgressBar  
    android:id="@+id/progress_bar" />  
  
<TextView  
    android:id="@+id/text_empty_state"  
    android:text="Нет сообщений.\nПотяните вниз для обновления." />
```

Основной контент по-прежнему отображается через RecyclerView:

```
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/recycler_view_messages"  
    tools:listitem="@layout/item_message" />
```

Экран ленты теперь поддерживает функцию pull-to-refresh, ручное обновление через FAB, правильную индикацию процесса загрузки и отображение пустого состояния. Это делает использование ленты более удобным и интуитивно понятным для пользователей.

Экран профиля также был визуально обновлен и приведен к Material-стилю. Для отображения основной информации используется карточка:

```
<com.google.android.material.card.MaterialCardView  
    android:id="@+id/card_profile">
```

Поля ввода имени и статуса обернуты в TextInputLayout:

```
<com.google.android.material.textfield.TextInputLayout  
    android:id="@+id/layout_name">  
  
<com.google.android.material.textfield.TextInputLayout  
    android:id="@+id/layout_status">
```

Такое оформление улучшает читаемость формы, визуально разделяет секции профиля и делает ввод данных более современным и удобным.

Появился полноценный экран настроек. Для переключения темы используется карточка и Material-переключатель:

```
<com.google.android.material.card.MaterialCardView  
    android:id="@+id/card_theme">  
  
<com.google.android.material.switchmaterial.SwitchMaterial  
    android:id="@+id/switch_theme" />
```

Такой подход гарантирует единообразие пользовательского опыта для настроек приложения и гармонирует с визуальным оформлением других экранов.

Карточка сообщения была полностью переработана и приведена к современному Material Design. В каждом элементе отображается аватар пользователя, имя, заголовок и текст сообщения, а также временная метка. Добавлена интерактивная кнопка «лайк», которая меняет свое состояние при нажатии. Само сообщение оформлено в виде MaterialCardView с закруглёнными углами и небольшим подъёмом, что визуально отделяет его от других сообщений. Такая структура улучшает читаемость, делает интерфейс более наглядным и интуитивным для пользователя, а также повышает интерактивность списка.

3 Демонстрация интерфейса приложения

В данном разделе представлены скриншоты разработанного приложения, демонстрирующие основные экраны и функциональные возможности. Интерфейс приложения выполнен в соответствии с принципами Material Design и обеспечивает интуитивно понятное взаимодействие с пользователем.

3.1 Экран ленты сообщений

На данном экране отображается список сообщений, полученных из сети и сохраненных в локальной базе данных. Каждое сообщение представлено в виде карточки Material Design с аватаром пользователя, именем, заголовком, текстом сообщения и временной меткой. Реализована возможность обновления списка через жест pull-to-refresh и плавающую кнопку. Также присутствует функция лайков с визуальной индикацией состояния.

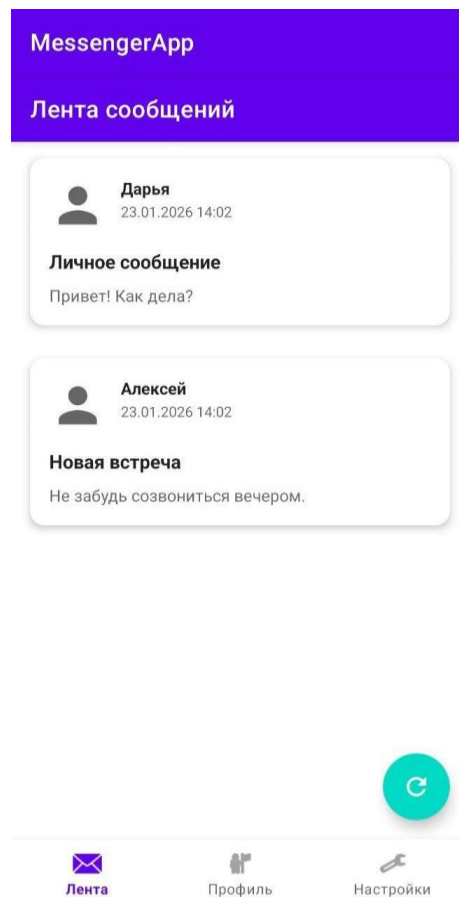


Рисунок 9 - Экран ленты сообщений

3.2 Экран настроек

Экран настроек позволяет пользователю переключаться между светлой и темной темой оформления. Переключатель выполнен с использованием компонента `SwitchMaterial` из библиотеки `Material Design`. Состояние темы сохраняется в `ViewModel` и автоматически восстанавливается при повороте экрана или перезапуске приложения.

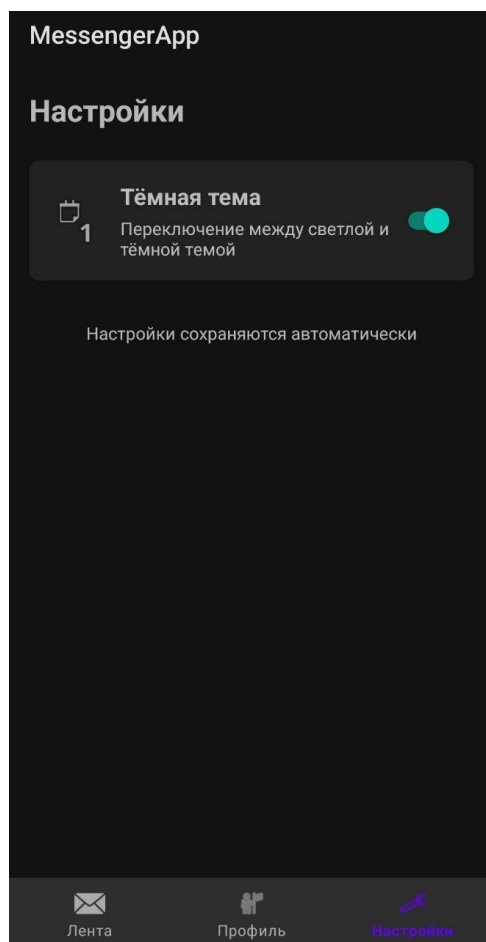


Рисунок 10 - Экран настроек (темная тема)

На следующем изображении представлен тот же экран настроек в светлой теме. Переключение между темами происходит мгновенно с сохранением всех пользовательских данных. Интерфейс адаптирован под оба режима оформления и обеспечивает комфортное использование в любых условиях освещения.

Настройки



Настройки сохраняются автоматически

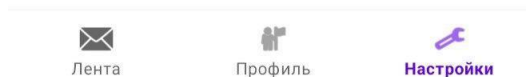


Рисунок 11 - Экран настроек (светлая тема)

3.3 Экран профиля пользователя

Экран профиля отображает информацию о текущем пользователе: имя и статус. Реализована возможность редактирования данных профиля через текстовые поля с материальным дизайном (`TextInputLayout`). Кнопки сохранения позволяют зафиксировать изменения, которые хранятся в `ViewModel` и автоматически восстанавливаются при изменении конфигурации устройства.

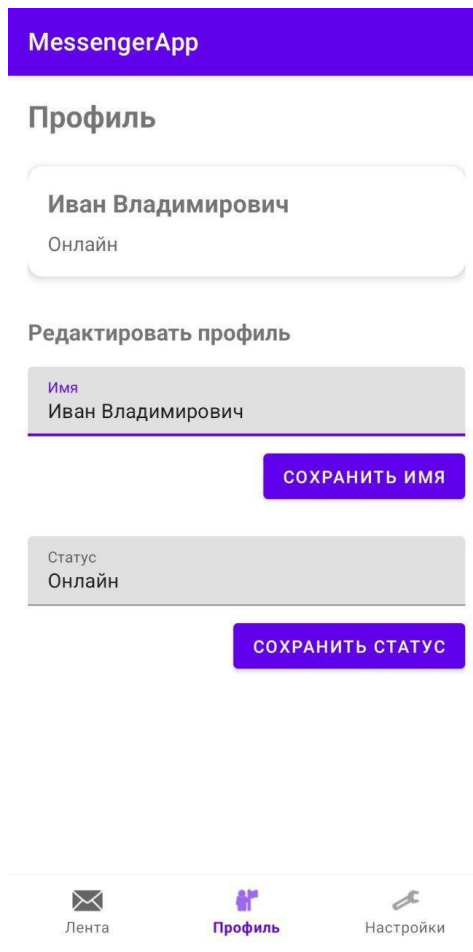


Рисунок 12 - Экран профиля пользователя

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта было разработано полнофункциональное Android-приложение для обмена сообщениями, демонстрирующее применение современных подходов к мобильной разработке и использование актуальных компонентов Android Jetpack.

Основным результатом работы стала реализация мобильного клиента с архитектурой MVVM, которая обеспечила четкое разделение ответственности между компонентами приложения. Применение ViewModel и LiveData позволило создать реактивный пользовательский интерфейс, автоматически обновляющийся при изменении данных. Использование Room обеспечило надежное локальное хранение данных и возможность работы приложения в офлайн-режиме. Интеграция Retrofit и корутин позволила реализовать эффективную работу с сетевыми API без блокировки основного потока.

Особое внимание в проекте было уделено пользовательскому опыту. Интерфейс приложения выполнен в соответствии с принципами Material Design с использованием современных компонентов: CardView для отображения сообщений, FloatingActionButton для быстрого доступа к функциям обновления, Bottom Navigation для интуитивной навигации между разделами. Реализована поддержка темной темы оформления, что повышает комфорт использования приложения в различных условиях освещения.

Внедрение WorkManager обеспечило надежную фоновую синхронизацию данных с учетом системных ограничений, таких как наличие сети и уровень заряда батареи. Механизм уведомлений информирует пользователя о поступлении новых данных, даже когда приложение находится в фоновом режиме.

Разработанное приложение полностью соответствует поставленным функциональным и нефункциональным требованиям всех четырех лабораторных работ. Код структурирован, документирован и готов к дальнейшему расширению функциональности. Применение архитектурных паттернов и современных инструментов разработки делает приложение масштабируемым и удобным в сопровождении.

Полученные в ходе выполнения курсового проекта практические навыки включают работу с жизненным циклом Android-компонентов, управление состоянием приложения, асинхронное программирование с использованием корутин, проектирование архитектуры приложения, работу с сетевыми API и локальными базами данных, а также создание современного пользовательского интерфейса. Эти компетенции являются ключевыми для профессионального Android-разработчика и могут быть применены при создании коммерческих мобильных приложений.

Таким образом, цель курсового проекта достигнута, все поставленные задачи выполнены. Разработанное приложение является завершенным программным продуктом, демонстрирующим владение современным стеком технологий Android-разработки и готовность к решению практических задач в области мобильной разработки.